786

Q-Learning

Syed Yusuf Hasan

1. Q learning is

i) Model free - does not need to know
   ⇒ transition probabilities $P(s'|s,a)$
   ⇒ reward function $R(s,a)$

ii) Off policy learns the optimal policy while behaving with different exploratory policy.

iii) Value based - learns a value function the derives policy from it.

2. Backbone (Mathematical)

Q learning assumes environment is MDP Markov Decision Process defined by tuples
   S ⇒ set of states
   A ⇒ set of actions
   $P(s'|s,a)$ ⇒ transition probability
   $R(s,a)$ ⇒ reward function
   $\gamma \in [0,1)$ ⇒ discount factor

No.

Date

Q learning convergences guarantees rely on MDP assumption.

Markov property : future depends only on current state not history.

$P(S_{t+1} \mid s_t, a_t)$

$= P(S_{t+1} \mid s_0, a_0, s_1, a_1 \ldots s_t, a_t)$

3. Return function, Discounting function, Value Function

3.1 Return (G_t) (is a function)

Total discounted reward from time t onwards

$G_t = r_{t+1} + \gamma \cdot r(t+2) + \gamma^2 \cdot r(t+2)^2 \ldots$

$= \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1}$

Neelgagan

No.

Date

Why discount ($\gamma < 1$) ?

→ Mathematically to keeps sum finite when episodes are unbounded.

→ Behavioural when

γ controls how far sighted agent is.

$\gamma = 0$ ⇒ only immediate reward

$\gamma = 1$ ⇒ values long term payoff.

3.11. State value function $V^{\pi}(s)$

i.e. expected return starting from s following policy π

$$V^{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

policy — return function — all states

Neelgagan

No.

Date

3.1.1.1) Action value function $Q^{\pi}(s,a)$

// expected return starting from s taking action a following policy π

$$Q^{\pi}(s,a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

// optimal value functions:

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

$$Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

$$V^*(s) = \max_{a} Q^*(s,a)$$

Bellman Optimality Equation for Q^*

for optimal action value function i.e Q^*

$$Q^*(s,a) = E\left[r_t + \gamma \cdot \max_{\{a'\}} Q^*(s',a') \mid s,a \right]$$

here s = current state

a = action taken in state s

r = immediate reward recieved

s' = next state

γ = discount factor $\in [0, 1)$

$\max_{\{a'\}} Q^*(s',a')$ - the best possible value achievable from next state

$E[\cdot \mid s,a]$ - expectation over environment's stochastic transitions & rewards

The equation is 'optimal' because it uses max instead of policy weighted sum.

∴ Q learning is off policy.

Neelgagan

No.

Date

Q learning — Update rule

old value → $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot$ ← learning rate

old value ↓ $[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$

immediate reward — temporal difference

TD target ← how wrong we were

old value

$\alpha = 0 \Rightarrow$ learns nothing

$\alpha = 1 \Rightarrow$ discard old estimate

$\alpha \in (0,1) \Rightarrow$ blend of old & new

Convergence Conditions

ie

Q-Learning converges to Q^* with probability = 1 if

→ Every (s,a) pair is visited infinitely

→ learning rate α satisfies Robins Monro conditions

→ MDP is finite ie finite states & actions.

Neelgagan

No.

Date

5. Exploration vs Exploitation

try new things ↙ ↘ use what it knows

5.1 ε - Greedy

with probability ε
→ choose a random action
otherwise
→ $\text{argmax}_a Q(s,a)$

property
→ simple, guarantees ∞ visitation

→ common schedule,
constant ε
∴ exponential decay $\varepsilon = \frac{1}{1+t}$

→ Downside is:
explores uniformly

Neelgagan

No.

Date

5.11) Softmax / Boltzmann Exploration

$P(a|s) = \exp(Q(s,a)/\tau) / \sum_b \exp(Q(s,b)/\tau)$

$\tau \Rightarrow$ Temperature controls randomness

→ explores proportional to value
∴ better than greedy

5.111) Optimistic Initialisation

initialise Q value high
∴ agent is disappointed
when it does not get high reward
∴
encourages exploring all actions naturally.

→ works only once

No.

Date

5.iv) UCB upper confidence bound

$$a = \text{argmax}_a \left[Q(s,a) + c \cdot \text{sqrt}\left(\frac{\ln N(s)}{N(s,a)} \right) \right]$$

⇒ Balances value & uncertainty

→ Deterministic

→ no ε parameter

5.v) Thompson Sampling i.e Bayesian

Maintain a distribution over Q values per action

& act greedily on samples

No.

Date

6. Temporal Difference learning (TD)

Q learning is TD method. //UOI

3 families of value estimation

3.1. Monte carlo

Update ⇒ full return of G_t

Variance ⇒ High

Bias ⇒ 0

ii) TD(0) ie $\alpha = 0$

update ⇒ one step target
$r + \gamma V'(s')$

Variance Low

Bias Some ie bootstraping

iii) TD(α) $\alpha = [0, 1)$

Variance & Bias both ↓↓ tunable

Neelgagan

No.

Date

Bootstrapping ⇒ using an estimate $V(s')$
to
update another estimate $V(s)$

TD converge faster than Monte Carlo
but
introduces bias

Neelgagan

No.

Date

overall Q learning algorithm

Step 1. Initialize Q Table

Create a table Q
with one entry per (state, action) pair
& set initial values.

(options)

		effect
zero	$Q(s,a) = 0$	neutral pair, simplest
optimistic	$Q(s,a) \equiv$ large +ve	encourages exploration ∴ agent is disappointed
pessimistic	$Q(s,a) \equiv$ large -ve	Rarely used
small random (preferred)	$Q(s,a) \sim N(0, \sigma^2)$	Breaks symmetry when actions otherwise identical

Neelgagan

No.

Date

Step 2 Choose an action ie
Behaviour policy ε-greedy

Given current state s
pick action a
using behaviour policy - ε-greedy

probability

1 - ε a = argmax_a Q(s,a) ⇒ exploit

ε a = random action ⇒ explore

alternative to ε-greedy

i) softmax / Boltzmann
ii) UCB
iii) thompson sampling

Neelgagan

No.

Date

Step 3 Act & observe

1. send action a to environment
2. Receive back
   r - ie immediate reward
   s' - next state
   done - whether the episode terminates

```
r, s', done = env.step(a)
```

Here environment may be

→ Deterministic
   ie same (s, a) → same (r, s')
   everytime

→ stochastic same (s, a) → random (r, s')

'done' flag ⇒ for correct target computation at next step.

```
td_target = r + γ . max Q(s') . (1 - done)
```

Neelgogan

No.

Date

Step 4. Update toward TD target

↳ this step is
↳ heart of Q learning
↳ actual learning happens

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\right]$$

Decomposed as:

$$\text{TD target} = r + \gamma \cdot \max_{a'} Q(s',a')$$

$$\text{TD error} = \text{TD target} - Q(s,a)$$

$$\text{New } Q = Q(s,a) + \alpha \cdot \text{TD error}$$

best possible next action

$Q(s,a)$ converges towards Q^*
ie
Bellman target

$$Q^*(s,a) = E\left[r + \gamma \cdot \max_{a'} Q^*(s',a') \,\middle|\, s,a\right]$$

Neelgagan

No.

Date

Two edge cases :

1) Terminal state s'

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r - Q(s,a)]$$

no bootstrapping
because
max $Q(\text{terminal}, .) = 0$

2) Non terminal state

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Neelgagan

No.

Date

step 5. Advance & repeat

→ set $s \leftarrow s'$

→ go back to step 2

→ loop until done

→ optional decay ε

decay

ε decay — high ε — explore
low ε — exploit

α decay - ie satisfy Robbins Monro conditions

$\sum \alpha_t = \infty$ ie keep learning
$\sum \alpha_t^2 < \infty$ ie variance shrink to zero

in practice

constant α is common in practice

converges only to a neighbourhood of q^*
not
exactly to q^*.

Syed Anwar Haider